

Speculative RAG: Latency-Quality Trade-offs in Multi-Draft Retrieval

Soyoon Park (sp4412), Rupeet Kaur (rk3408), Hsuan-Ting Lin (hl3930)

Abstract—Speculative Retrieval-Augmented Generation (Speculative RAG) promises latency and accuracy gains over standard RAG in multi-GPU settings [1]. In this project, we observe if such gains can be obtained on a single-GPU hardware with memory constraint. We recreate the Speculative RAG pipeline on a single A100 (80GB) and quantify its latency and quality trade-offs compared to the standard RAG baseline. Moreover, we evaluate Speculative RAG with three backends (No optimization, NF4 quantization, vLLM). All instances are evaluated on the TriviaQA dataset. We observe that the paper’s accuracy and latency results depend on (a) parallel drafter execution on multiple GPUs and (b) a fine-tuned drafter. Without these, the standard RAG was found to be faster and more accurate. Yet, Speculative RAG was observed to have a massive memory advantage over the baseline RAG. This work provides profiling of Speculative RAG under realistic hardware settings and identifies drafter fine-tuning and retrieval acceleration as the critical path to recover the results reported by the paper.

I. INTRODUCTION

Large language models (LLMs) have excelled at generating plausible-looking responses to user questions. Yet, the accuracy of the response tend to suffer for questions that require dense, obscure, or up-to-date information. Retrieval-Augmented Generation (RAG) has gained attention as a solution to address the problem—RAG mitigates errors by including external documents to the context. However, RAG introduces its own concern of increased latency due to the increase in input length to the LLMs.

Wang et al. proposes the Speculative RAG framework [1] to address the latency problem in the standard RAG while boosting the accuracy. Inspired by Speculative Decoding, Speculative RAG uses smaller specialist drafter to generate high-quality draft answers from distinct subset of retrieved documents and generalist verifier LM to incorporate the most promising draft to the final answer. Speculative RAG is claimed to have up to 12.97% accuracy gain while reducing latency by 50.83% compared to conventional RAG systems.

We recreate the Speculative RAG system with the reduced hardware constraint of single-GPU setting and compare its accuracy, latency, memory usage, and throughput with the standard RAG under the same hardware resource. Moreover, we compare the latency-quality tradeoff of applying different optimization techniques (NF4 quantization, vLLM) on the Speculative RAG compared to the unoptimized baseline. Thus, through this comparative study, we not only examine the likelihood of recovering the claimed accuracy and latency benefits of Speculative RAG under reduced hardware capacity, but also explore the impact of optimization options that may further improve the latency.

II. LITERATURE REVIEW

Retrieval-Augmented Generation (RAG) allows AI models access to given information, so the model can answer the question using external evidence instead of only answering based on memorized knowledge. This also improves the factual accuracy [2]. However, the model needs to read through many retrieved passages in a long prompt, which slows down the inference process. Moreover, the model sometimes ignores important details in the middle of a long document [3].

To solve this problem, some researchers do extra training for models, teaching them to actively filter out unimportant information, or critique the work they generate [4, 5]. But retraining is expensive, and sometimes the AI model forgets its original information. To avoid expensive training, some researchers use a lightweight evaluator tool to give retrieved documents a relevance score [6]. But this evaluator is too small to have enough reasoning ability to connect multiple pieces of evidence together.

To speed up the generation of answers without expensive retraining, some researchers use a method called speculative decoding. Instead of letting a large model generate every token one by one, they make a smaller model draft possible next tokens, and let the larger model verify them in parallel [7, 8]. Because there is an additional memory cost when running a separate draft model, some researchers make the target model create drafts internally or use extra decoding heads to reduce overhead [9, 10].

Speculative RAG combines these ideas with retrieval. This method does not give all documents to the model at once. It splits retrieved documents into smaller subsets, then generates the answer and rationale drafts from each subset. Next, it uses a verifier to select the best answer [1]. Our implementation follows this idea with limited memory. We use one Mistral-7B-Instruct model for both drafting and verification.

III. METHODOLOGY

A. Models and Data Description

We used the TriviaQA validation split and the DPR Wikipedia passage corpus to evaluate our system. Then, we used facebook/contriever-msmarco to embed the questions. Next, we built a FAISS IndexFlatIP index from the passage embeddings and performed our retrieval over that. Our standard RAG system retrieved the top-10 passages and further combined these passages into a single prompt for mistralai/Mistral-7B-Instruct-v0.1.

We used the same retrieval backend for speculative RAG. The difference is that the retrieved passages were divided

into multiple document subsets. Next, we passed each subset to a drafter model, which further generated an answer and rationale. We used the Mistral-7B-Instruct model to serve as both the drafter and verifier because our memory was limited. We did not use the fine-tuned MDrafter or Mixtral-8x7B verifier like the original paper did, so our accuracy was also affected.

B. Training and Profiling Methodology

We did not train additional models in our reproduction. The drafter was used zero-shot with a prompt, and this prompt required the format ‘Rationale’ and ‘Response’. Next, we retrieved the top documents for each question, then clustered them using their embeddings from the Contriever. Then, we sampled m subsets with k documents per subset. After this, the verifier used the drafter log probability, self-consistency score, and self-reflection score to score each draft and select the answer with the highest score.

We used the normalized containment metric to measure exact match accuracy. The normalized containment metric is commonly used in Self-RAG and TriviaQA-style evaluations. Moreover, we measured latency separately for retrieval, subset sampling, drafting, verification, and end-to-end execution. Besides, we reported p50 and p95 latency, total pipeline runtime, token counts, throughput, and peak GPU memory. Furthermore, we used PyTorch Profiler for HuggingFace no-optimization and NF4 runs, and used Nsight Systems for vLLM profiling.

C. Performance Tuning Methodology

We compared three inference configurations. The first one is a no-optimization HuggingFace baseline, and it uses standard `generate()` calls. This is inefficient since each draft introduced repeated CPU scheduling and GPU kernel launch overhead. The second one is NF4 4-bit quantization through bitsandbytes. This configuration reduced GPU memory usage a lot. However, it introduced dequantization overhead and did not effectively use Tensor Cores. So, we found that the latency increased and the accuracy dropped. The third one is vLLM with PagedAttention and continuous batching. Under this configuration, the GPU can process drafts more efficiently since m drafter prompts for each question were submitted together. We found that this also reduced the drafting latency and total pipeline runtime, so we think vLLM is the most effective optimization. Since vLLM accelerated drafting, retrieval became the main remaining bottleneck.

IV. EXPERIMENTAL RESULTS

A. Experiment setup

Three configurations were tested:

- **No optimization:** standard HuggingFace `generate()` with greedy decoding.
- **NF4 quantization:** `BitsAndBytesConfig` (`load_in_4bit = True`, `bnb_4bit_quant_type = ‘nf4’`).

- **vLLM:** vLLM server with PagedAttention and continuous batching; all m Drafter prompts dispatched as a single async call per question.

Standard RAG and Spec RAG No opt each were run on 1000 TriviaQA validation samples (seed=3) for head-to-head comparison. Speculative RAG without optimization, NF4 quantization and vLLM were run on 100-sample with $m=5$, $k=2$ for comparison. The m -sweep ($m \in \{5, 10, 15, 20\}$, $k=2$) was run under the vLLM backend for hyperparameter analysis.

B. Before and After Optimization

1) *No-optimization:* Without any inference optimization, the Drafter runs in standard FP16 HuggingFace mode. The PyTorch Profiler trace shows an average step time of 7.82ms, with CPU Exec accounting for 57.5% of that wall time (Fig. 5a). GPU utilization is 26.6%, and estimated SM efficiency is 21.0%, which is below the hardware’s potential. The dominant operators are `aten::mm` (25.9% of host self time) and `aten::mul` (16.4%), showing standard FP16 matmul-heavy autoregressive decode. Peak GPU memory allocation is 15797 MB.

Since we used a single GPU, the serial `generate()` calls for each draft create substantial idle time between kernel launches. Because the Drafter processes one draft at a time without continuous batching, the GPU finishes a kernel, returns to Python, dispatches the next token, and repeats. The CPU launch overhead dominates for small-batch autoregressive decoding. End-to-end p50 latency per query is 10681 ms, with the drafting stage alone taking 3937 ms at p50.

2) *After NF4 Quantization:* NF4 quantization reduces the Drafter’s weights from FP16 to 4-bit. A dequantization step is performed on the fly during each linear layer forward pass. Peak GPU memory drops dramatically to 6129 MB, a clear win over the no optimization setup. However, latency actually increases (see Fig. 3a). As evident from the PyTorch Profiler (see Fig. 6), `MatMul4Bit` (bitsandbytes’ quantized kernel) becomes the dominant operator, with 38.2% of host self time, while on-device duration is reported as zero. GPU utilization actually increases to 48.8% and SM efficiency to 45.8% as compared to No opt. However, Tensor Core utilization for `MatMul4Bit` is 0%. Accuracy drops from 54% (No opt) to 48% EM under NF4, consistent with documented 4-bit quality loss on instruction-tuned 7B models.

3) *After vLLM Continuous Batching:* vLLM restructures the drafting step fundamentally. All m draft prompts for a given question are submitted as a single batched request, and KV cache is managed via PagedAttention and schedules tokens across drafts using continuous batching. The p50 draft latency falls to 1932 ms from the 3937 ms baseline. End-to-end p50 is 9131 ms (see Fig. 3a).

The total pipeline time is 1157s, compared to 1518s for No opt, showing around 25% speedup. Continuous batching eliminates the per-step CPU dispatch overhead seen in the No opt profiler, and PagedAttention removes the KV cache fragmentation that is present in the No opt case (see Fig. 1b for

TABLE I: Optimization Comparison (100 Samples, $m=5$, $k=2$)

Metric	No opt	NF4	vLLM
EM (100 samples)	54%	48%	47%
p50 e2e latency (ms)	10,681	12,881	9,131
p50 draft latency (ms)	3,937	7,662	1,932
p50 verify latency (ms)	64	98	81
p50 retrieve latency (ms)	6,609	5,140	7,050
Total pipeline (s)	1,518	2,007	1,157
Peak GPU mem alloc. (MB)	15,797	6,129	470
Throughput (questions/s)	0.06	0.045	0.082

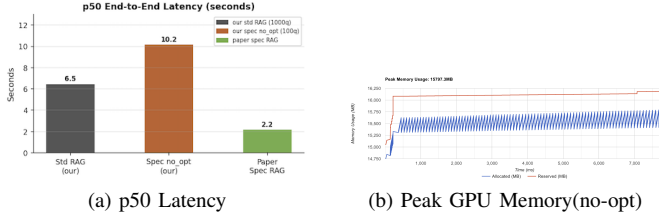


Fig. 1: Performance Comparison & No-opt Profiler Trace

the sawtooth allocation pattern). The GPU operations visible to nsys (see Fig. 5b) show 99.8% of GPU execution time as `cudaMemcpyAsync` Host-to-Device transfers. The 330 μ s average memcopy operations show the scheduling overhead.

Table I summarizes the results across all three backends.

C. Comparative study analysis

1) *Standard RAG (1,000 Samples) vs. Speculative RAG No opt (1,000 Samples) vs. Paper*: Standard RAG achieves 80% EM on TriviaQA at a p50 end-to-end latency of 6,017 ms. Speculative RAG No opt achieves 62% EM at a p50 latency of 9,180 ms.

On accuracy, Standard RAG outperforms Speculative RAG due to implementation constraints. The paper reports 74% accuracy on TriviaQA using MVerifier-8x7B with a fine-tuned MDrafter-7B (trained on 40059 instruction-tuning examples). The Drafter here is zero shot: it has never been trained to produce the structured (answer, rationale) format the Verifier’s log probability scoring depends on. The Verifier is also Mistral-7B rather than Mixtral-8x7B, reducing the quality of the final draft selection pass. We did not use Mixtral-8x7B due to memory constraints since nsys traces for even a single sample were of size in gigabytes.

On latency, Standard RAG p50 e2e is 6017 ms vs. Spec RAG’s 9180 ms. Both systems are heavily retrieval-dominated (p50 retrieve $\approx$ 5350 to 5820 ms), but Spec RAG adds extra drafting time. Standard RAG generation latency is only 165 ms at p50 (generating a short final answer from a single prompt); Spec RAG’s draft latency is 3733 ms (generating 5 full rationale and answer pairs). The paper shows a latency advantage for Spec RAG since it launches 5 parallel Drafter replicas on 5 GPU endpoints. We could not avail the true parallelism since the experiments were run on a single device without batching. Standard RAG also saturates VRAM at 73066 MB (out of 80 GB) because its long combined prompt fills KV

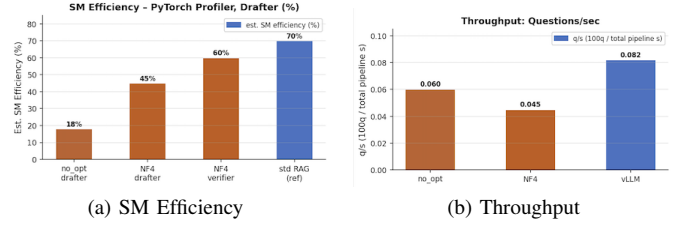


Fig. 2: Before/After Optimization: SM Efficiency, Throughput

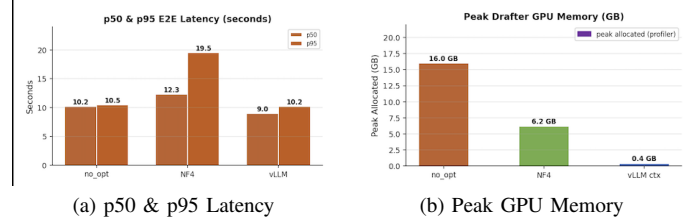


Fig. 3: Before/After Optimization: Latency, Memory

cache. Spec RAG’s per-subset prompts are much shorter, with peak memory at 15797 MB.

2) 100 Samples: No opt vs. NF4 vs. vLLM ($m=5$, $k=2$):

Accuracy: Rationale generation quality degrades slightly due to NF4 quantization, and since the Verifier’s log probability scoring is sensitive to rationale quality, errors compound. **Latency**: vLLM delivers the best latency. NF4 is slower than No opt despite being the quantized model since there is a round trip from Python for `MatMul4Bit` compute with 0 Tensor Core utilization. The result shows the effect of slower drafting negating memory savings. **Memory**: vLLM delivers a clear memory win. **SM Utilization**: There is a large scheduling overhead for no-opt (58% CPU Exec). NF4 improves apparent GPU activity (49%) because dequantization is GPU work, but throughput is lower since it is low-arithmetic-intensity work with 0% Tensor Core usage. vLLM benefits from continuous batching. With drafting accelerated, retrieval now constitutes around 77% of total e2e latency under vLLM (p50 retrieve = 7,050 ms vs. p50 e2e = 9,131 ms). Retrieval has become the clear new bottleneck.

3) *Speculative RAG across $m \in \{5, 10, 15, 20\}$ (vLLM, $k=2$)*: More diverse document subsets give the Verifier more candidates to evaluate, improving the chance of finding the correct answer, hence the EM gain from $m=5$ to $m=20$ (even though marginal). However, it plateaus due to subset redundancy at larger m (see Fig. 4). There is an increase in latency from $m=5$ to $m=20$ due to the $4\times$ increase in drafts. $m=20$ achieves the best accuracy (52%) at an 8,201 ms p50 e2e. However, the gain is marginal (+5 EM) and it adds cost to latency. So $m=5$ offers the optimal choice without costing latency.

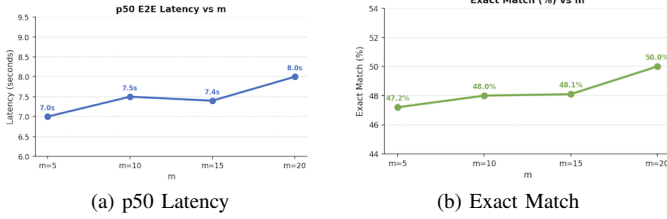


Fig. 4: Hyperparameter m sweep

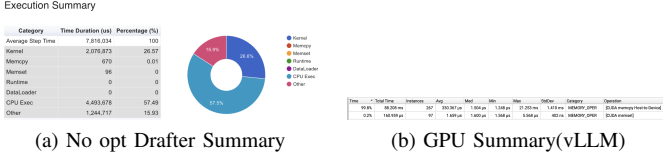


Fig. 5: PyTorch Profiler & Nsys Trace

V. DISCUSSION

A. Interpretation of Experimental Results

vLLM outperforms all configurations in speculative RAG. It reduces draft latency by 51%, total pipeline time by 24%, without sacrificing accuracy, owing to continuous batching and PagedAttention. Due to per token dequantization overhead in bitsandbytes, NF4 pays a draft latency penalty. Replacing bitsandbytes with a GPTQ or AWQ backend that supports fused dequant matmul with Tensor Core usage would likely narrow the latency gap. The paper’s fine tuned MDrafter-7B is what leads Speculative RAG to surpass Standard RAG and therefore the discriminativeness of the Verifier’s log probability scoring depends on a tuned drafter to produce good (answer, rationale) pairs. LoRA fine tuning on TriviaQA examples would be expected to improve accuracy.

For Standard RAG, SM utilization meets the proposal’s $\geq 60\%$ target because it runs a single large prompt through one model continuously. For Speculative RAG, short per draft calls, each with smaller batch sizes, leaves the GPU under-utilized between dispatches. Retrieval is the large bottleneck. No optimization applied here targets retrieval. FAISS GPU would be expected to close it, however it was not used in our experiments due to memory constraints. The m -sweep confirms vLLM’s batching efficiency does not degrade as m grows. Batching the Verifier scoring calls at high m would allow $m=20$ or beyond to remain competitive on latency.

B. Limitation

Our single-GPU (A100 80GB) hardware setup proved to be the root cause of most experimental decisions that diverged from the Speculative RAG paper. For one, we could not exploit parallelism in the drafter phase as was done in the paper with multiple GPUs; serial execution over m drafting iterations is the largest source of latency disadvantage over measured RAG baseline. For a fair comparison, we could have approximated the post-retrieval latency as (draft latency / m) + verifier latency. Moreover, due to memory limitation, we could

not load FAISS-GPU along vLLM, hence we used FAISS-CPU instead with a flat index. Such inefficiency would have contributed to the retrieval stage being the dominating latency bottleneck (about 77% of total latency in vLLM test case). However, since this was the case for all experiments, it does not affect the comparative latency.

Regarding latency, it is also to be noted that our baseline RAG was run with batch size 32 instead of processing one question at a time as the Speculative RAG does, which gave unfair latency advantage. The paper also implies that their baseline measurement was done by processing questions serially in Section 4.5: “[t]o simulate real-world application scenarios, we process cases individually without batching.” [1]

Regarding accuracy, we observed two main anomalies: not only did Speculative RAG had lower accuracy than standard RAG, but also that baseline RAG was measured to have higher accuracy than was reported in the paper. There are two main explanations for this. First, our drafter model was not fine-tuned as was in the paper, which likely has the biggest impact in the accuracy drop in Speculative RAG. Second, we used a simplified EM metrics which inflates the reported accuracy compared to the EM metrics used in the paper. While this does not explain the relative accuracies between our experiment setups, it explains why our baseline accuracy exceeds that of the paper.

C. Future Work

In aggregate, the highest impact path toward matching the paper’s results is:

- 1) LoRA fine tune the Drafter for structured (answer, rationale) generation. Even a few thousand TriviaQA examples (as opposed to 40k in paper) would show a meaningful improvement in accuracy.
- 2) Use vLLM for both Drafter and Verifier with batched scoring calls.
- 3) Accelerate retrieval via FAISS GPU with a non-flat index.

Quantization adds value only if a larger Verifier (13B or 8x7B) is introduced.

If multiple-GPU setup was possible, we can achieve true parallel drafter execution by deploying m drafter replicas across multiple GPUs, which is expected to bring the latency reduction as claimed by the paper.

VI. CONCLUSION

We re-implemented Speculative RAG under a single-GPU hardware constraint and compared it against the baseline standard RAG over the metrics of latency, accuracy, memory, and throughput. Additionally, the compared three inference backends—No optimization, NF4 quantization, vLLM—to explore optimization options for Speculative RAG.

From the sets of comparative experiments, we discovered the following:

- 1) vLLM is the best optimization backend that achieved 51% draft latency reduction and 24% total pipeline

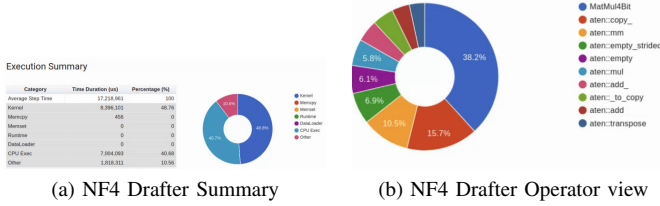


Fig. 6: PyTorch Profiler overview: NF4 drafter

speedup with minimal accuracy cost over no optimization.

- 2) NF4 quantization saves memory but hurts latency.
- 3) Standard RAG outperformed Speculative RAG in accuracy (80% vs. 62%) primarily because the drafter was not fine-tuned, weakening the effectiveness of the following verifier’s selection mechanism.
- 4) Retrieval was measured to be the biggest latency bottleneck, not generation.
- 5) Speculative RAG has a considerable memory advantage over standard RAG (15.8 GB vs. 73 GB peak).

From the observations, we learn that the Speculative RAG paper’s claimed benefits are largely conditional to the available hardware resource. The latency benefit of speculative RAG is difficult to claim under single-GPU setting without parallelism. The accuracy gains depend heavily on the fine-tuned drafter, followed by using a larger general model for the verifier. LoRA fine-tuning the drafter model and moving to FAISS-GPU retrieval are the two clearest paths toward recovering the paper’s results. This work lays the groundwork of instrumentation and profiling for measuring those improvements.

VII. IMPORTANT LINKS

- **Wandb:** https://wandb.ai/soyoon-columbia-university/hpml_rag
- **GitHub:** <https://github.com/soyoon-cu/speculative-rag>.git

VIII. AI USE POLICY

We used AI assistance as described: Explored and clarified concepts related to Speculative RAG and other ideas described in the paper [1]. Debugged the final report for LaTeX syntax errors after writing the report ourselves. Debugged code for retrieving embeddings instead of directly fetching unembedded passages using indices. Assistance in Linking the GCP bucket for embedded passage retrieval. Assistance in handling failed imports. Explored the use of Nsight Systems for the first time. Tools used: Gemini, ChatGPT. Sections affected: Retriever, Final Report, vLLM. We verified correctness by: Successful imports and results. By submitting this project, we confirm that the analysis, interpretations, and conclusions are our own, and that any AI assistance is fully disclosed above.

REFERENCES

- [1] Zilong Wang et al. “Speculative RAG: Enhancing Retrieval Augmented Generation Through Drafting”. In: *International Conference on Learning Representations (ICLR)*. 2025.
- [2] Patrick Lewis et al. “Retrieval-augmented generation for knowledge-intensive NLP tasks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020.
- [3] Nelson F Liu et al. “Lost in the middle: How language models use long contexts”. In: *Transactions of the Association for Computational Linguistics* (2024).
- [4] Hongyin Luo, Qingxiu Sun, et al. “Sail: Search-augmented instruction learning”. In: *arXiv preprint arXiv:2305.18848* (2023).
- [5] Akari Asai et al. “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection”. In: *International Conference on Learning Representations (ICLR)*. 2024.
- [6] Shi-Qi Yan et al. “Corrective Retrieval Augmented Generation”. In: *arXiv preprint arXiv:2401.15884* (2024).
- [7] Yaniv Leviathan, Matan Kalman, and Yossi Matias. “Fast inference from transformers via speculative decoding”. In: *International Conference on Machine Learning (ICML)*. 2023.
- [8] Charlie Chen et al. “Accelerating large language model decoding with speculative sampling”. In: *arXiv preprint arXiv:2302.01318* (2023).
- [9] Jun Zhang, Jue Wang, et al. “Draft & verify: Lossless large language model acceleration via self-speculative decoding”. In: *arXiv preprint arXiv:2309.08168* (2023).
- [10] Tianle Cai et al. “Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads”. In: *International Conference on Machine Learning (ICML)*. 2024.